

Trabajo de Conceptos Generales: Estructura de Datos

Nombre

David Alejandro Rosales Sarria

Instructor

Luis Fernando Sánchez Pérez

Ficha

3169892

Servicio Nacional de Aprendizaje SENA

Centro De Tecnología De La Manufactura Avanzada CTMA

Análisis y Desarrollo De Software

2025

Solución de Guía 0

Actividad 1: Sistemas Numéricos

Decimal	Binario	Octal	Hexadecimal
5050	1001110111010	11672	13BA
4096	1000000000000	10000	1000
4567	1000111000111	10707	11D7
3800	111011011000	7330	EE8
1098	10001001010	2112	44A
1024	10000000000	2000	400
2048	100000000000	4000	800
2512	100111010000	4720	9D0
1970	11110110010	3632	7B2
680	1010101000	1250	2A8
512	1000000000	1000	200
300	100101100	454	12C
256	100000000	400	100
128	10000000	200	80
190	10111110	276	BE
64	1000000	100	40
56	111000	70	38
10	1010	12	A
780	1100001100	1414	30C
912	1110010000	1620	390

360	101101000	550	168
-----	-----------	-----	-----

Consultas

Operaciones Aritméticas en Sistemas Numéricos

Los sistemas numéricos, incluyendo el binario, octal y hexadecimal, permiten realizar las operaciones aritméticas básicas:

- **Suma:** Se realiza de manera similar a la suma decimal, pero teniendo en cuenta el valor de la base. Por ejemplo, en binario, $1 + 1 = 10$ (2 en decimal).
- **Resta:** Similar a la resta decimal, pero considerando la base. En binario, se pueden utilizar métodos como el complemento a dos para facilitar la operación con números negativos.
- **Multiplicación:** Se puede realizar mediante sumas repetidas o con algoritmos similares a la multiplicación decimal, adaptados a la base del sistema numérico.
- **División:** Similar a la división decimal, pero adaptada a la base del sistema numérico.

Conversiones entre Sistemas Numéricos

Conversiones Comunes

se describen las conversiones más comunes entre sistemas numéricos:

- **Binario a Octal:**
 1. Agrupar los dígitos binarios en grupos de tres, comenzando desde la derecha.
 2. Convertir cada grupo de tres dígitos binarios a su equivalente octal.
- **Octal a Binario:**
 1. Convertir cada dígito octal a su equivalente de tres dígitos binarios.
- **Octal a Decimal:**
 1. Multiplicar cada dígito octal por su valor posicional (potencias de 8).
 2. Sumar los resultados.
- **Decimal a Octal:**
 1. Dividir el número decimal entre 8 de forma sucesiva.
 2. Tomar los residuos de cada división, en orden inverso, para obtener el número octal.
- **Decimal a Hexadecimal:**
 1. Dividir el número decimal entre 16 de forma sucesiva.

2. Tomar los residuos de cada división, en orden inverso, para obtener el número hexadecimal. Si un residuo es mayor que 9, representarlo con la letra correspondiente (A=10, B=11, C=12, D=13, E=14, F=15). iii
- **Hexadecimal a Decimal:**
 1. Multiplicar cada dígito hexadecimal por su valor posicional (potencias de 16).
 2. Sumar los resultados.

Bit, Nibble y Byte

- **Bit:** Es la unidad mínima de información en informática. Representa un dígito binario (0 o 1).
- **Nibble:** Es un grupo de 4 bits.
- **Byte:** Es un grupo de 8 bits. Es una unidad común para representar caracteres, números pequeños y otras formas de datos.

Actividad 2: Almacenamiento de Datos

Estructura de un Registro de Memoria

Un registro de memoria es una pequeña cantidad de almacenamiento de alta velocidad dentro de la CPU. Los registros se utilizan para contener datos e instrucciones que la CPU está procesando activamente.

Estructura Básica

Aunque las estructuras específicas pueden variar entre diferentes arquitecturas de CPU, un registro de memoria típicamente incluye:

- **Nombre o Identificador:** Cada registro tiene un nombre único (por ejemplo, AX, BX, CX, DX en la arquitectura x86) que el programador o el hardware utiliza para referirse a él.
- **Tamaño:** El tamaño de un registro determina la cantidad de datos que puede contener. Los tamaños comunes son 8 bits, 16 bits, 32 bits y 64 bits.
- **Tipo de Datos:** Los registros pueden contener diferentes tipos de datos, como enteros, números de punto flotante o direcciones de memoria.
- **Funcionalidad:** Algunos registros tienen propósitos especiales. Por ejemplo, el registro contador de programa (PC) contiene la dirección de la siguiente instrucción a ejecutar, y el registro acumulador (AC) se utiliza para almacenar resultados intermedios de operaciones aritméticas y lógicas.

Tipos de Registros

- **Registros de Datos:** Se utilizan para almacenar operandos y resultados de operaciones.
- **Registros de Direcciones:** Contienen direcciones de memoria de datos o instrucciones.

- **Registros de Propósito General:** Pueden ser utilizados para una variedad de propósitos por el programador.
- **Registros de Propósito Específico:** Tienen funciones predefinidas, como el contador de programa o el registro de estado.

iv

Creación de un Puntero en Lenguajes de Programación

Un puntero es una variable que almacena la dirección de memoria de otra variable. Permite acceder y manipular directamente los datos almacenados en esa dirección de memoria. La sintaxis para crear punteros varía entre lenguajes de programación:

Ejemplo en C/C++

```
int variable = 25; // Declara una variable entera
int *ptr;         // Declara un puntero a un entero
ptr = &variable;  // Asigna la dirección de 'variable' a 'ptr'

// 'ptr' ahora contiene la dirección de memoria de 'variable'
```

Sistema de Archivos

Un sistema de archivos es la estructura lógica y la organización que un sistema operativo utiliza para almacenar, administrar y acceder a archivos y directorios en un dispositivo de almacenamiento (como un disco duro, SSD o unidad USB). Define cómo se nombran los archivos, dónde se almacenan, cómo se recuperan y cómo se actualizan.

Funciones Clave

- **Organización de Datos:** Estructura los datos en archivos y directorios para facilitar el acceso.
- **Asignación de Espacio:** Administra el espacio de almacenamiento disponible en el dispositivo.
- **Acceso a Archivos:** Proporciona métodos para crear, leer, escribir, modificar y eliminar archivos.
- **Seguridad:** Controla el acceso a los archivos mediante permisos y privilegios.
- **Integridad de Datos:** Asegura la consistencia y confiabilidad de los datos almacenados.

Sistemas de Archivos Más Utilizados

- **NTFS (New Technology File System):** El sistema de archivos principal utilizado por los sistemas operativos Windows modernos.
- **APFS (Apple File System):** El sistema de archivos predeterminado para macOS, iOS y otros sistemas operativos de Apple.
- **Ext4 (Fourth Extended Filesystem):** Un sistema de archivos ampliamente utilizado en sistemas operativos Linux.

- **FAT32 (File Allocation Table 32):** Un sistema de archivos más antiguo, pero todavía utilizado para unidades flash USB y tarjetas de memoria debido a su compatibilidad.
- **XFS:** Un sistema de archivos de alto rendimiento que se utiliza comúnmente en sistemas Linux para grandes volúmenes de almacenamiento.

v

Almacenamiento de Datos en un Disco Duro

Un disco duro (HDD) almacena datos digitalmente en superficies magnéticas.

Componentes Clave

- **Platos:** Discos circulares hechos de un material no magnético (generalmente aluminio o vidrio) y recubiertos con un material magnético.
- **Cabezales de Lectura/Escritura:** Dispositivos que leen y escriben datos magnéticamente en los platos.
- **Actuador:** Brazo que mueve los cabezales de lectura/escritura a través de la superficie de los platos.
- **Motor del Husillo:** Hace girar los platos a altas velocidades.

Proceso de Almacenamiento

1. **Organización:** Los datos se organizan en pistas concéntricas y sectores en cada plato.
2. **Magnetización:** Los cabezales de lectura/escritura magnetizan pequeñas áreas en la superficie del plato para representar bits (0 o 1). La dirección de la magnetización determina si el bit se lee como 0 o 1.
3. **Rotación:** El motor del husillo hace girar los platos, permitiendo que los cabezales de lectura/escritura accedan a diferentes partes del disco.
4. **Movimiento del Cabezal:** El actuador mueve los cabezales de lectura/escritura a la pista correcta para leer o escribir datos.

Actividad 3: Estructuras de Datos

Tipos de Datos en Lenguajes de Programación

Los tipos de datos definen el tipo de valor que una variable puede almacenar. Aquí hay algunos tipos de datos comunes:

Tipos de Datos Primitivos

- **Enteros:** Representan números enteros (por ejemplo, -1, 0, 1, 100).
 - Ejemplos: `int`, `short`, `long`, `byte`
- **Números de Coma Flotante:** Representan números con decimales (por ejemplo, 3.14, -0.5, 2.0).
 - Ejemplos: `float`, `double`

- **Booleanos:** Representan valores de verdadero o falso.
 - Ejemplo: `bool` o `boolean`
- **Caracteres:** Representan un solo carácter (por ejemplo, 'a', 'Z', '\$').
 - Ejemplo: `char`

Tipos de Datos Compuestos

- **Cadenas:** Representan secuencias de caracteres (por ejemplo, "Hola", "Mundo").
 - Ejemplo: `string`
- **Arrays (Arreglos):** Colecciones de elementos del mismo tipo.
- **Estructuras:** Colecciones de elementos de diferentes tipos.
- **Punteros:** Variables que almacenan la dirección de memoria de otra variable.

Tipos de Datos en Bases de Datos

Los tipos de datos en bases de datos son similares a los de los lenguajes de programación, pero con algunas variaciones y especificidades:

- **Numéricos:**
 - Enteros: `INT`, `BIGINT`, `SMALLINT`, `TINYINT`
 - Decimales: `DECIMAL`, `NUMERIC`
 - Punto Flotante: `FLOAT`, `REAL`, `DOUBLE`
- **Cadena de Caracteres:**
 - Longitud fija: `CHAR`
 - Longitud variable: `VARCHAR`, `TEXT`
- **Fecha y Hora:**
 - Fecha: `DATE`
 - Hora: `TIME`
 - Fecha y Hora: `DATETIME`, `TIMESTAMP`
- **Booleanos:** `BOOLEAN`
- **Otros:**
 - Datos binarios: `BINARY`, `VARBINARY`, `BLOB`
 - Tipos de datos espaciales: `GEOMETRY`, `POINT`, `LINESTRING`
 - JSON: Para almacenar datos en formato JSON.

Collation en Bases de Datos

En bases de datos, un "collation" es un conjunto de reglas que definen cómo se comparan y ordenan los datos de tipo carácter. Determina aspectos como:

- **Sensibilidad a mayúsculas y minúsculas:** Si "a" es igual a "A".
- **Sensibilidad a acentos:** Si "á" es igual a "a".
- **Orden de clasificación:** El orden alfabético de los caracteres.
- **Comparación de caracteres Unicode:** Cómo se comparan diferentes codificaciones de caracteres.

El collation es importante para:

- **Consultas de comparación:** Afecta los resultados de las operaciones de comparación (por ejemplo, `WHERE nombre = 'Juan'`).
- **Ordenamiento:** Determina el orden de los resultados en las cláusulas `ORDER BY`.
- **Indexación:** Influye en cómo se indexan y buscan los datos de tipo carácter.

Ejemplos de Collation

Diferentes bases de datos soportan diferentes collations. Algunos ejemplos comunes incluyen:

- `utf8_general_ci`: Collation de MySQL, no distingue entre mayúsculas y minúsculas, ni acentos.
- `utf8_bin`: Collation de MySQL, distingue entre mayúsculas y minúsculas, y compara los caracteres de forma binaria.
- `SQL_Latin1_General_CP1_CI_AS`: Collation de SQL Server, no distingue entre mayúsculas y minúsculas, distingue entre acentos.

El collation se puede especificar a nivel de servidor, base de datos, tabla o incluso columna.

